Procedural Programming

Part 2: External Modules

Programs can be separated into separate files to increase readability and maintainability.

Creating External Programs

As programs grow, keeping all code in one file can make them harder to read and maintain.

External modules allow related functions and classes to be moved into separate files.

The main program can then focus on the overall flow of the program while supporting code is kept elsewhere.

This also makes useful code easier to reuse in other programs.

Creating External Modules

A **module** is simply a Python file that contains code you want to use somewhere else.

Functions, classes, and variables can all be stored inside a module.

The module file can then be imported into another Python program.

```
# math_tools.py

def double_number(number):
    return number * 2
```

Importing External Modules

Python can import specific functions or classes from another module using **`from ... import ...`**.

This places the imported item directly into the current program's namespace.

You can then use it without writing the module name first.

```
# main.py

from math_tools import double_number



result = double_number(5)
print(result)
```

Module Being Imported Example

```python
# main.py

from math_tools import double_number


result = double_number(5)
print(result)
```

```python
# math_tools.py

def double_number(number):
    return number * 2
```

Testing Modules with `if __name__ == "__main__"`

A module can contain test code without having that code run when the module is imported.

The condition

```
if __name__ == "__main__":
```

only runs when the file is executed directly.

This makes it useful for testing functions and classes inside their own module.

```
# math_tools.py

def double_number(number):
    return number * 2

if __name__ == "__main__":
    print(double_number(5))
```

```
OUTPUT
10
```

One-Way Parameter Passing

One-way parameter passing means that a value is sent into a function so the function can use it.

The function receives the value through a parameter, but does not send a result back with **`return`**.

This is useful when the function only needs to perform an action.

```
def greet(name):
    print(f"Hello, {name}!")

greet("Alex")
```

OUTPUT

```
Hello, Alex!
```

Two-Way Parameter Passing

Two-way parameter passing means that data is sent into a function and a result is returned back to the calling code.

Parameters provide the input, while **`return`** provides the output.

This allows the result to be stored and used elsewhere in the program.

```
def double_number(number):
    return number * 2

result = double_number(5)
print(result)
```

OUTPUT

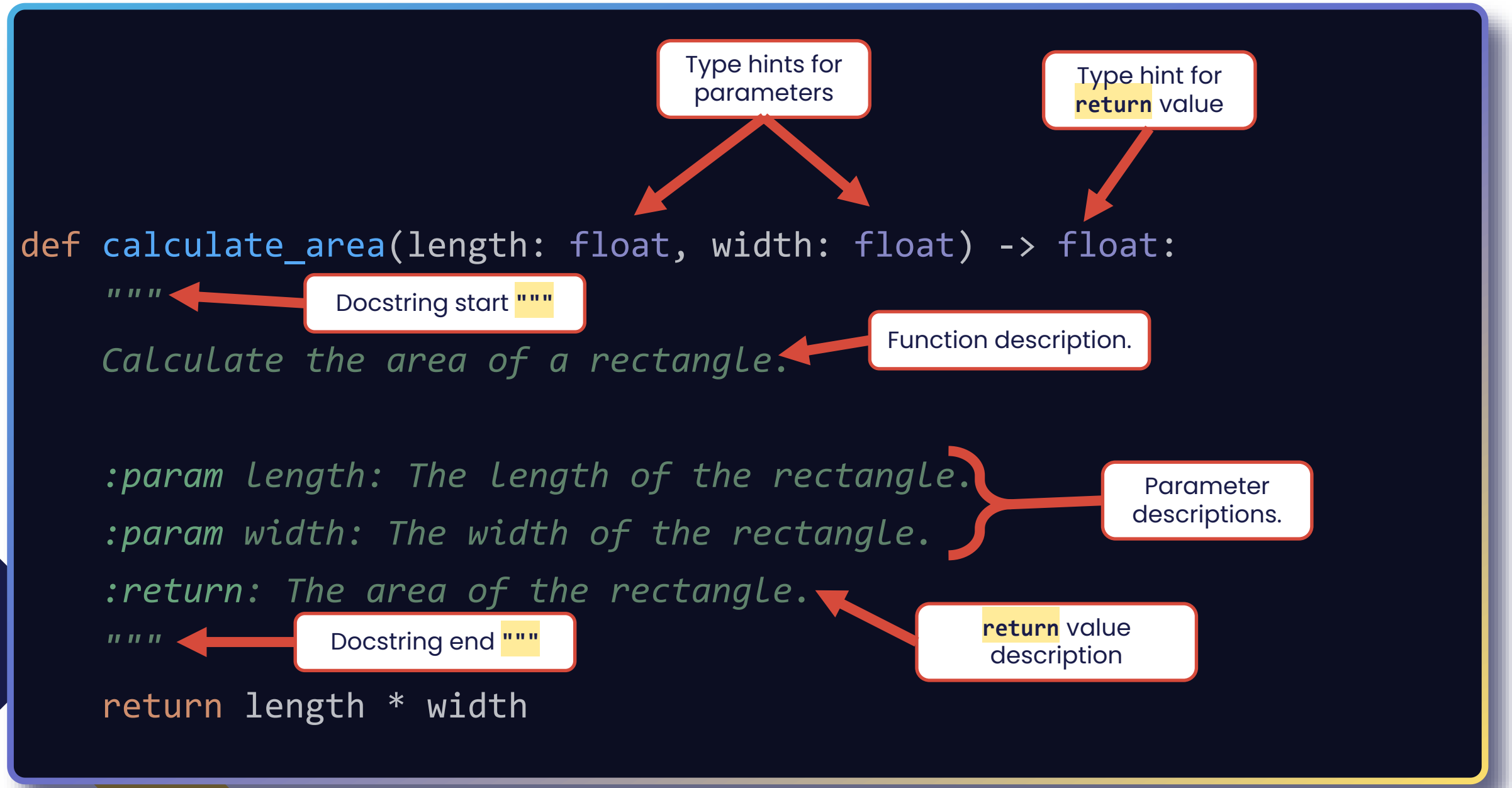
```
10
```

Documentation for External Modules

External modules should include clear documentation so another programmer can understand how to use them without reading all of the code.

Docstrings can describe what a module, function, or class does, what parameters it expects, and what values it returns.

Good documentation is especially important when a module is designed to be reused by other programs.


Type hints for parameters
Type hint for return value
def calculate_area(length: float, width: float) -> float:
"""
Docstring start """
Calculate the area of a rectangle.
Function description.
:param length: The length of the rectangle.
:param width: The width of the rectangle.
Parameter descriptions.
:return: The area of the rectangle.
"""
Docstring end """
return value description
return length * width